



Flutter

...

by M.T.I. Juan Luis Campos Barrera



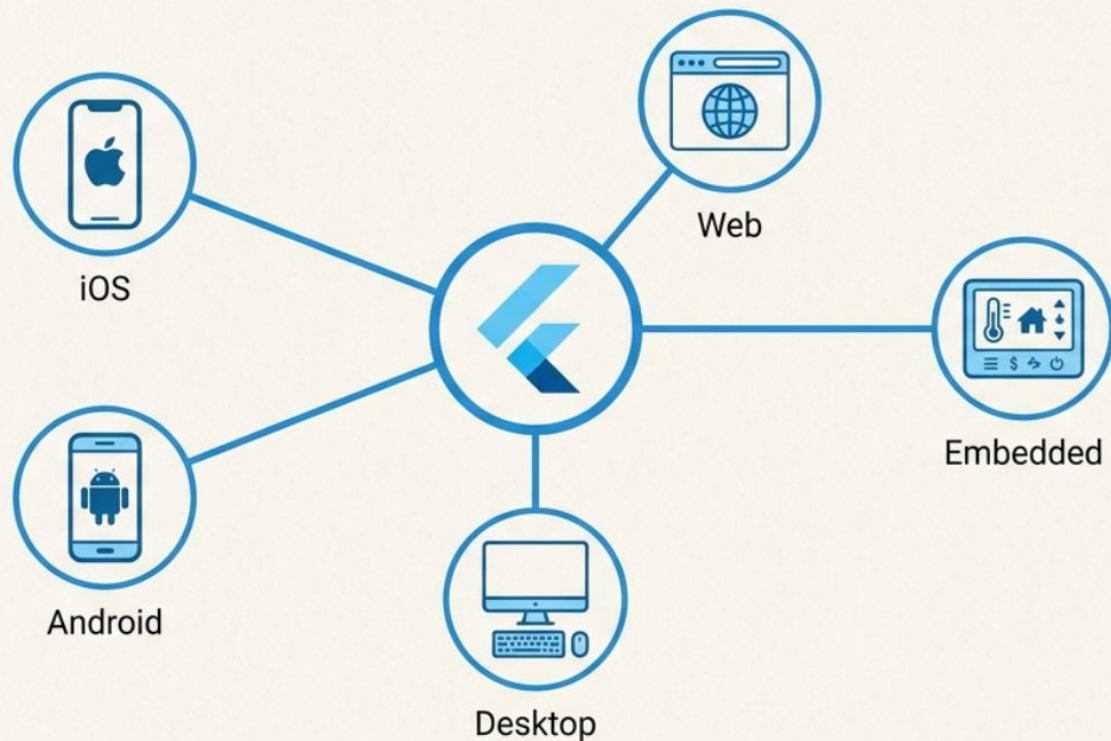
De la Lógica a la Interfaz: Fundamentos de Flutter y Dart

El motor, el chasis y la carrocería para el
desarrollo móvil multiplataforma.

BOTÓN

Escribir aquí...

La Promesa Multiplataforma



Un solo código

Desarrollo unificado para **iOS**, **Android**, **Web** y **Desktop** desde una única base de código.



Renderizado Nativo

Compila directamente a **código máquina (ARM/Intel)** o **JavaScript**, garantizando rendimiento fluido sin puentes intermedios.



Hot Reload

Iteración en milisegundos. Los cambios en el código se reflejan instantáneamente sin perder el estado de la aplicación.

Arquitectura Moderna: El Rol de la Presentación



¿Dónde encaja Flutter?

Actúa puramente como el rostro de la aplicación.

Consume APIs de distintos orígenes de forma transparente para el usuario final, gestionando únicamente el estado de la interfaz y la experiencia (UI/UX).

El Motor: Fundamentos de Dart

Dart es un lenguaje orientado a objetos, optimizado por Google para la creación de interfaces rápidas en cualquier plataforma.



Tipado Inferido vs Estricto

Capacidad de usar 'var' dinámico o definir tipos fijos de memoria ('int', 'String').



Ejecución Secuencial

Todo programa en Dart comienza invariablemente en la función void main().

```
// El punto de entrada principal  
void main() {  
  print('Hola, Flutter');  
}
```

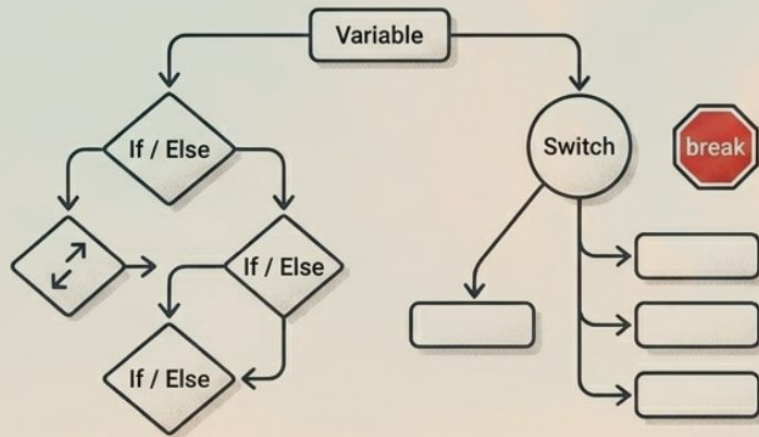
El inicio
absoluto de
la ejecución

Bloques de Datos: Tipos de Variables

Tipo	Sintaxis	Regla
String	<code>String name = 'Aris';</code>	Requiere comillas simples o dobles, 'S' mayúscula.
int / double	<code>int age = 31;</code>	Números enteros y decimales. También soportado por 'num'.
bool	<code>bool isHappy = true;</code>	Decisiones binarias. Solo admite true o false.
dynamic	<code>dynamic ex = 'Hola'; ex = 23;</code>	Peligroso: ignora la validación de tipos y rompe la trazabilidad.
final / const	<code>final String apodo = 'Pepe';</code>	Inmutables. No pueden reasignarse tras su creación.

Control de Flujo y Colecciones

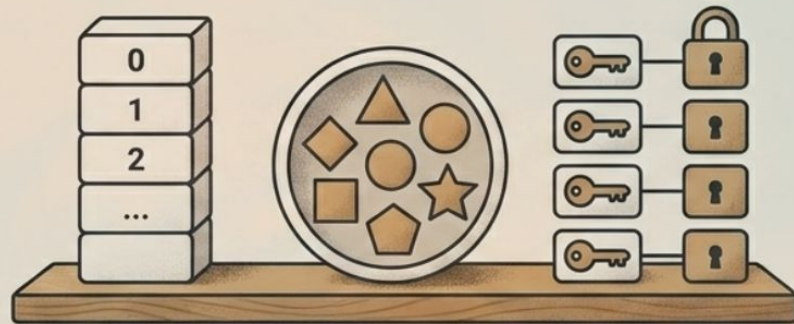
Lógica y Rutas



If / Else: Evaluaciones secuenciales dinámicas.

Switch: Selector directo para evaluar casos fijos.

Agrupación de Datos



List []

Colección ordenada por índice (comienza en 0). Permite duplicados.

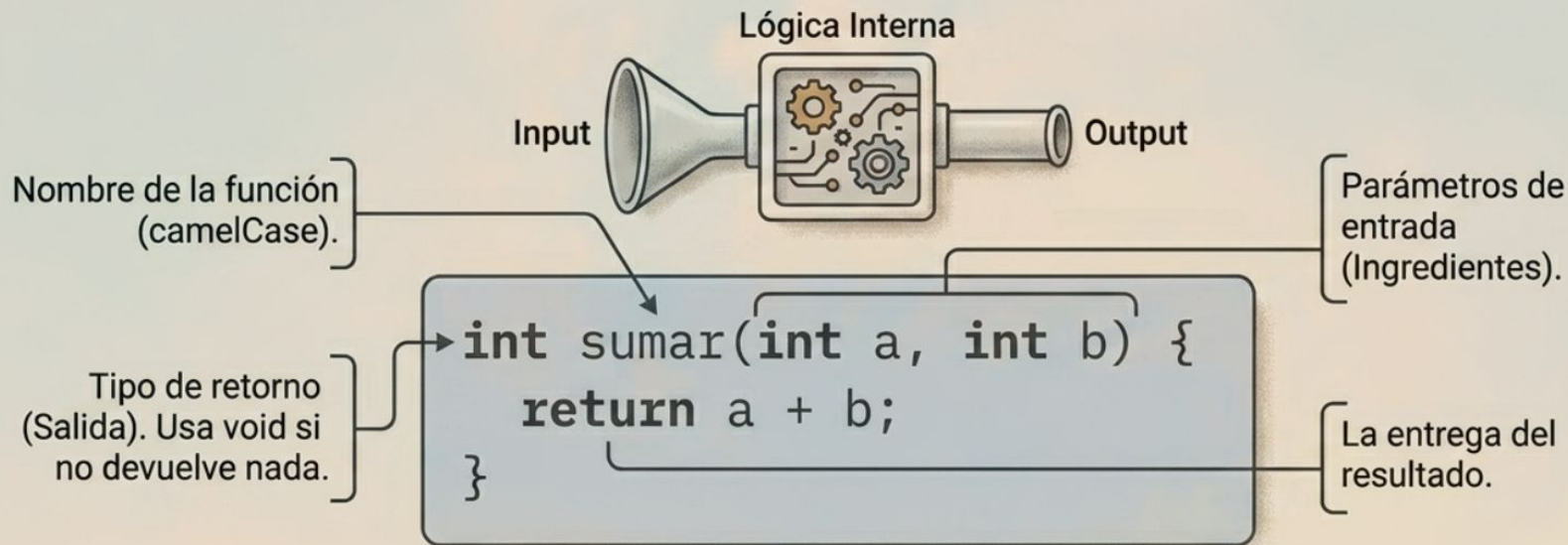
Set {}

Colección sin orden. Rechaza automáticamente datos duplicados.

Map {key: value}

Almacena pares vinculados. Acceso mediante la clave.

Encapsulación: Funciones y Parámetros



Parámetros Opcionales

Las funciones pueden usar corchetes `[]` o llaves `{}` para definir parámetros opcionales, permitiendo valores por defecto.

El Chasis: Todo es un Widget



Sin Separación

En Flutter, no hay separación entre código de vista (HTML/XML) y código lógico. La interfaz se construye ensamblando clases de Dart.

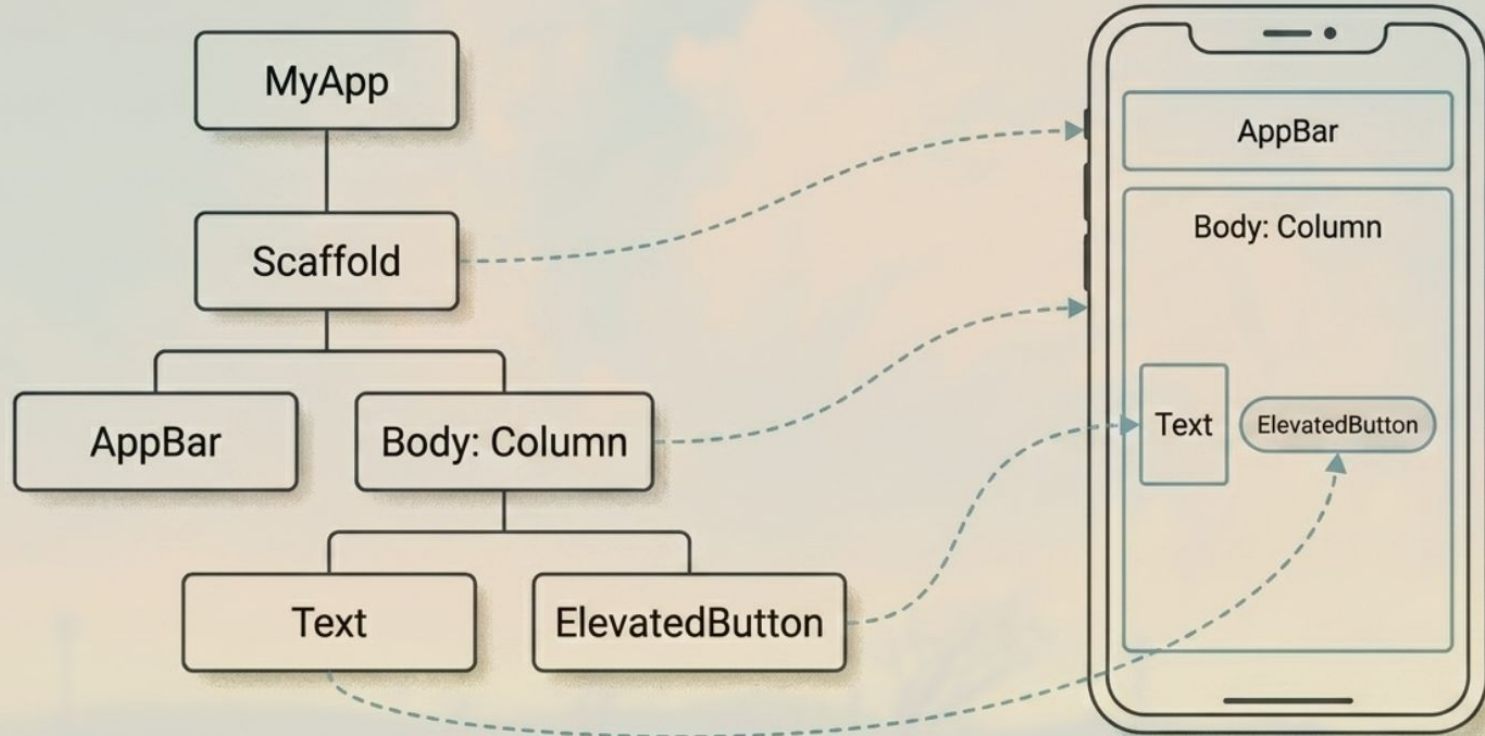
Todo es Estructural

Un texto es un Widget. Un botón es un Widget. El margen (padding) es un Widget. La pantalla completa es un Widget.

Anidación (Nesting)

Las interfaces complejas son simplemente el resultado de anidar Widgets simples dentro de otros Widgets, formando una jerarquía.

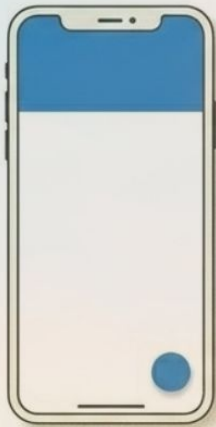
El Árbol de Widgets (Widget Tree)



Estructuras Espaciales: Organizando la Pantalla

Scaffold (El Esqueleto)

Estructura base. Soporta appBar, body y floatingActionButton.



Column (Flujo Vertical)

Apila múltiples Widgets hijos (children) de arriba hacia abajo.



Row (Flujo Horizontal)

Alinea múltiples Widgets hijos (children) de izquierda a derecha.



Componentes Clave de Interfaz

Text()



Atributo clave: **style: TextStyle(fontSize: 20)**

TextField()



Atributo clave: Permite la entrada de datos por parte del usuario.

Image.network()



Atributo clave:
fit: BoxFit.cover
(Carga imágenes desde una URL).

Botones



Atributo clave:
onPressed: () {}
(Ejecuta una acción. Si es null, se desactiva).

El Motor de la Reactividad: Stateless vs Stateful

	Stateless Widget	Stateful Widget
Definición	Estático e inmutable.	Dinámico y reactivo.
Metáfora Visual	 Un póster impreso en la pared.	 Un marcador digital deportivo.
Caso de Uso Ideal	Textos fijos, iconos decorativos, estructuras base.	Formularios, contadores, casillas de verificación, animaciones.
Ciclo de Vida	Solo requiere el método <code>build()</code> .	Requiere una clase auxiliar <code>State</code> y el uso del método <code>setState()</code> .

Anatomía de un Estado: El Ciclo setState

1. Interacción (UI)

El usuario toca un botón interactivo en la pantalla.



2. Mutación (Lógica)

Se altera una variable pura de Dart (ej. `counter++`).



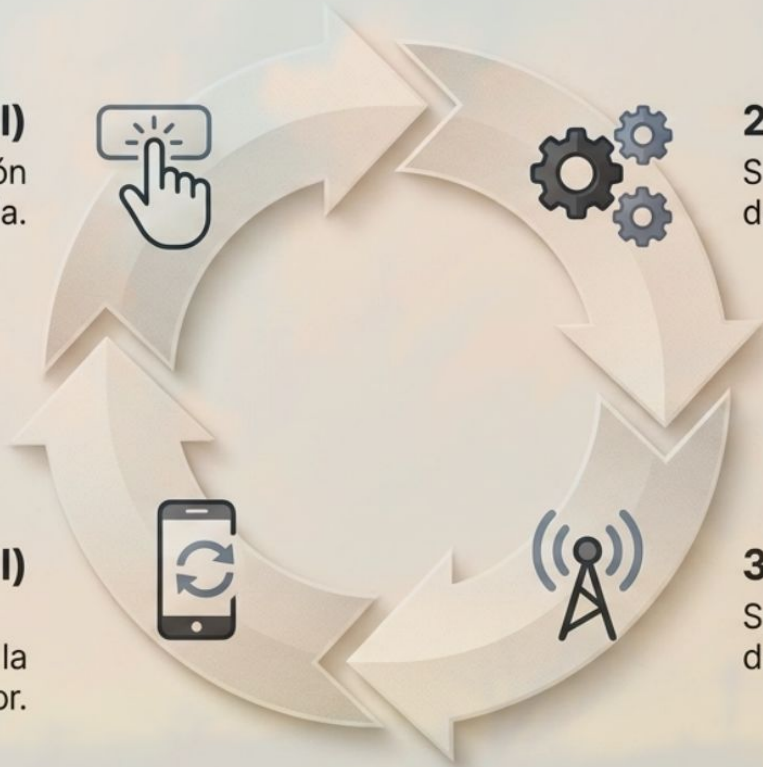
3. Notificación (Puente)

Se envuelve la mutación dentro del método `setState()`.



4. Reconstrucción (UI)

Flutter invalida la vista, ejecuta `build()` y redibuja la pantalla con el nuevo valor.



El Ensamblaje: Tu Primera App Funcional

El código y la interfaz gráfica no son mundos separados. En Flutter, el código es la interfaz.

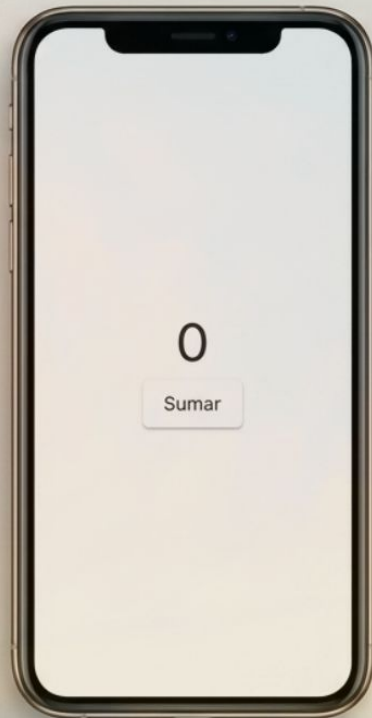
```
class MyScreen extends StatefulWidget {  
  int counter = 0;  
  void increment() {  
    setState(() {  
      counter++;  
    });  
  }  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Column(  
        children: [  
          Text('$counter'),  
          ElevatedButton(  
            onPressed: increment,  
            child: Text('Sumar')  
          )  
        ],  
      ),  
    );  
  }  
}
```

Variable tipada
(Dart puro)

Estructura reactiva
(Chasis)

Ciclo de actualización
(Motor)

Árbol de Widgets y
Layout espacial



Más Allá de lo Básico: El Siguiete Nivel

Navegación (Routing)

Pasar de una pantalla a otra no es magia. Usa `Navigator.push()` para apilar nuevas vistas (Widgets) sobre la pantalla actual, construyendo flujos de usuario completos.



Asincronía y APIs (Consumiendo Datos)

Usa el poder de Dart (`Future`, `async/await`) y paquetes HTTP para conectar Flutter a backends robustos (Django/Go), transformando datos JSON en vistas dinámicas.



Con las bases de Dart, la comprensión del Árbol de Widgets y el dominio del Estado, tienes todo lo necesario para empezar a construir para cualquier pantalla.